



TREG

By Anton Winkler

16th of March 2009

1

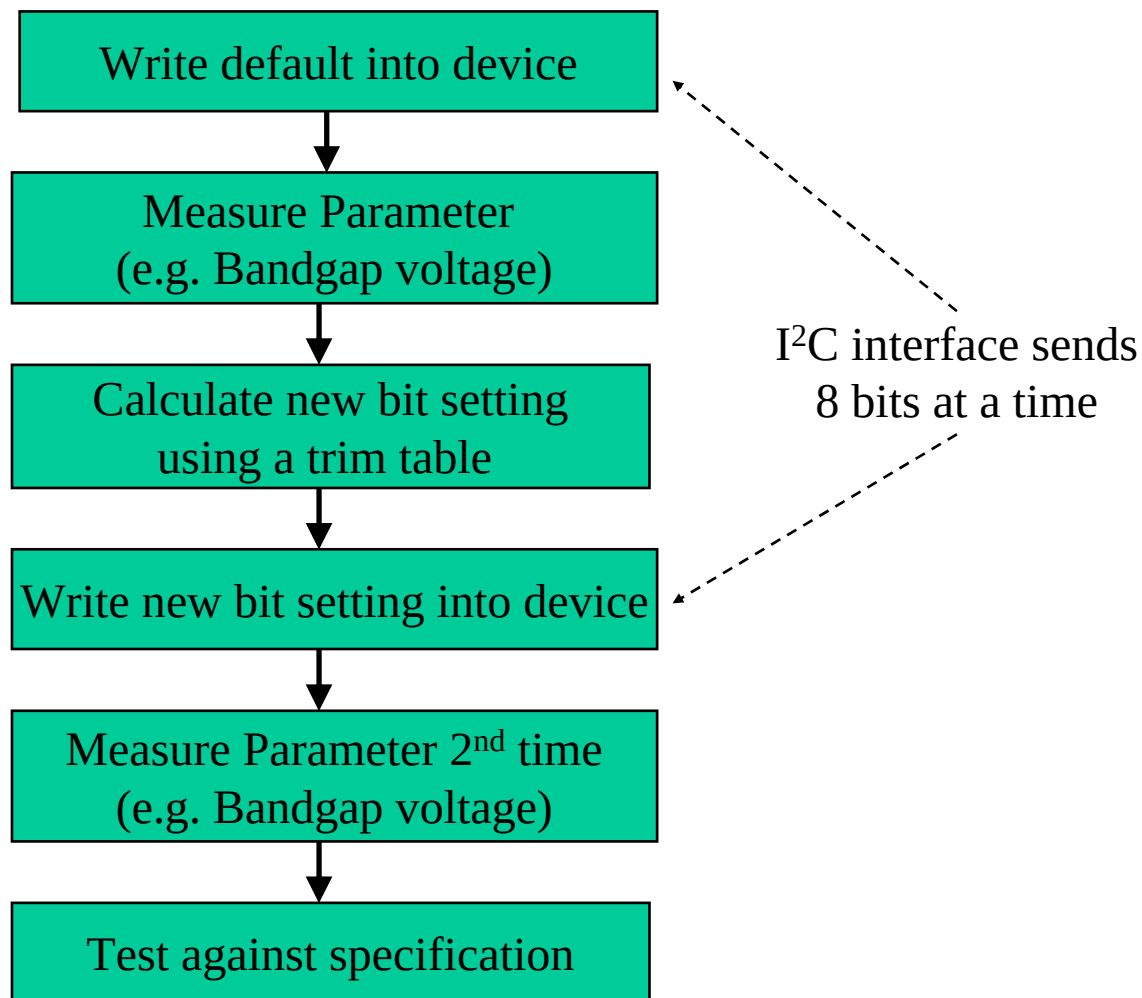


Agenda

- How do we trim a parameter
- What is TREG
- Basic Concept
- Trimming section
- Selection Bit section
- Assy section
- Assy Group section
- Setup of Class
- How to use
- Debug capabilities
- Adaptive Trim learning

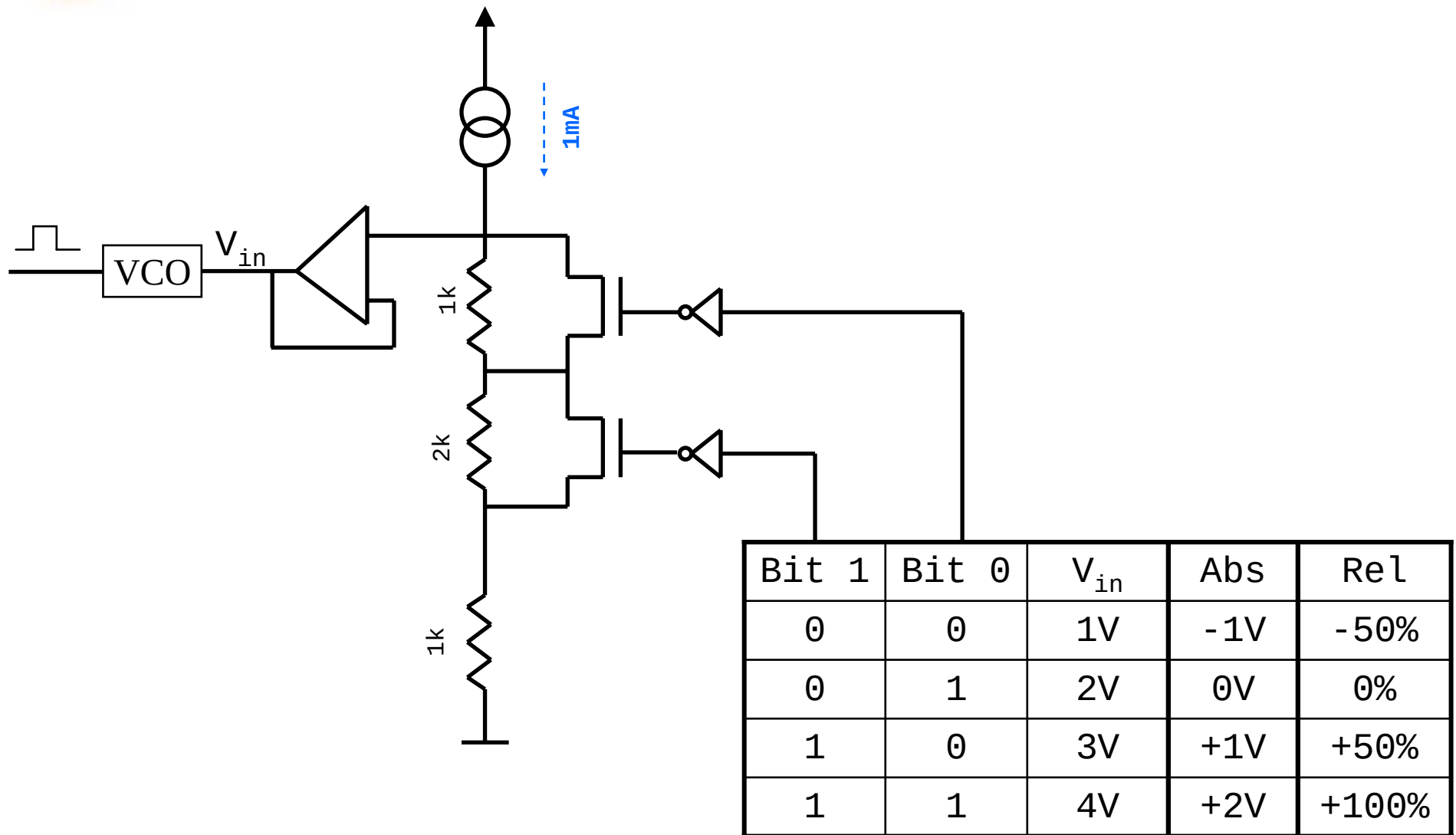


Trimming flow





Simple Trimming Circuit





Register Table

For some devices we have a couple of more bits then just two.....

I2C ADDR	QUICK I2C ADDR		ACCESS	INTERNAL										
W/R	W	R												
CUSTOMER														
00	00	80	R/W	mem_0	VLOW		STAT		BOOST		FAULT			
01	01	81	R/W	mem_1	IIN LIMIT		TIMER RESET		TERM EN		RESET	CHARGE_ENb	HZ_MODE	OPA_MODE
02	02	82	R/W	mem_2	VOREG								OTG_PL	OTG_EN
03 (3B)	03	83	R		VENDOR ID				PART NUMBER				REVISION	
			W ¹	test_mode_0	Enable TEST MODE		Enable QUICK I2C	Disable DEGLITCH	Enable EXT CLK	Disable SAFETY TMR	Use SOFT TRIM	Enable DIG TEST		
04	04	84	R/W	mem_4	---		VICHARGE		---		VITERM			
1: EN_TEST must be asserted														
TEST MODE														
07	07	87	R/W	test_mode_0	Enable TEST MODE		Enable QUICK I2C	Disable DEGLITCH	Enable EXT CLK	Disable SAFETY TMR	Use SOFT TRIM	Enable DIG TEST		
08	08	88	R/W	test_mode_1	Clear Mem	Reduce Currents					Force EEPROM Read	Burn EEPROM		
SOFT TRIM (Enable DIG TEST = 0)														
09	09	89	R/W	test_mem_0	BG				ENB_SAFETY_TIMER		TEST_ID			
0A	0A	8A	R/W	test_mem_1	IBUS_100MA						OSC			
0B	0B	8B	R/W	test_mem_2	IBUS_500MA						VREG			
0C	0C	8C	R/W	test_mem_3	GN_ICHG				OS_ICHG					
0D	0D	8D	R/W	test_mem_4	ENB_TMPREG	COMP_SELECT	ENB_3MHZ	ENB_BUCK_LSILIM	ENB_BUCK_HSILIM	ENB_BST_ILIM	OS_IBUS			
0E	0E	8E	R/W	test_mem_5	BST_ILIMLIN		LS_ILIM		ENB_BQ24150	ENB_CCM	ENB_PFM	ENB_TSHUT		
0F	0F	8F	R/W	test_mem_6	ENB_FIXED_RMP	ENB_BST_HSILIM	ENB_BST_LSILIM	TRIM_BUCK_HSILIM		TRIM_BST_HSILIM		EN_500MA		
DIGITAL TESTING (Enable DIG TEST = 1)														
09	09	89	R/W	test_mem_0	Force SDA			Force State	Controller State ^a					
0A	0A	8A	R/W	test_mem_1	Enable FAULT PULSE	Enable TIMER ALARM	Bist CONTROLLER	Bist DEGLITCH	Bist SHIFT OUT					
0B	0B	8B	R/W	test_mem_2	DG_uvlob	DG_bg_good	DG_valhd	DG_en_clk	DG_signal_in					
0C	0C	8C	R/W	test_mem_3										
0D	0D	8D	R/W	test_mem_4										
0E	0E	8E	R/W	test_mem_5										
0F	0F	8F	R/W	test_mem_6										
2: See table below														
MUX CONTROL (Enable DIG TEST = 0)														
10	10	90	R/W	mem[7:0]	STAT	OSC (15K)	State Match	Force State	Controller State ^a					
11	11	91	R/W	mem[15:8]	SLEEP	RECH	POORSRC	LOWV_BATMIN	CHOV_BATOV	BUSUVLOB	BUSOV_BSTOV	BATSHRT		
12	12	92	R/W	mem[23:16]	HSILIM	LSILIM	BTSTREF	PFM	BST_OCP	PWM	TSHUT	TERM		
13	13	93	R/W	mem[31:24]		SCL	SDA	OTG	I2C_STOP	I2C_RD_STR	I2C_WR_STR	I2C_START		
14	14	94	R/W	mem[40:32]	TMOD_BTSTREF	TMOD_IIN_300MA_SEL	TMOD_BG	TMOD_CMP_PWM	SOFTSTART	TMOD_HSD_RDS	TMOD_PRESET_FORCE	TMOD_PFM_FORCE		
15	15	95	R/W	mem[47:40]	TMOD_PWRSTG_FORCE		TMOD_QUIET	TMOD_LSD_RDS	TMOD_TREG_TRIGGER	TMOD_TSHUT	TMOD_VRCHDAC	TMOD_BSTCMP		
16	16	96	R/W	mem[55:48]	Disable PWM	Disable PWM_CFG	Disable SLEEP	Disable CHARGE						
17	17	97	R/W	mem[63:56]	Disable SINK	Disable OSC_LP	Disable OSC	Disable LINCHG	Disable FET_500	Disable FET_100	Disable DACS	Disable BOOST		



Trim and Selection Bits

EEPROM bits (with highlighted trim and selection bits)....

BG				ENB_SAFETY_TIMER		TEST_ID	
IBUS_100MA						OSC	
IBUS_500MA						VREG	
GN_ICHG				OS_ICHG			
ENB_TMPREG	COMP_SELECT	ENB_3MHZ	ENB_BUCK_LSILIM	ENB_BUCK_HSILIM	ENB_BST_ILIM	OS_IBUS	
BST_ILIMLIN		LS_ILIM		ENB_BQ24150	ENB_CCM	ENB_PFM	ENB_TSHUT
ENB_FIXED_RMP	ENB_BST_HSILIM	ENB_BST_LSILIM	TRIM_BUCK_HSILIM		TRIM_BST_HSILIM		EN_500MA

= trim parameter

= selection parameter



What do we have to deal with:

- Trim Parameter handling (e.g. oscillator)
- Selection Bit handling (e.g. Thermal shutdown)
- Register assembly and disassembly (interface)
- Register Groups (for multi- register devices)
- Multi-site capability



What is TREG

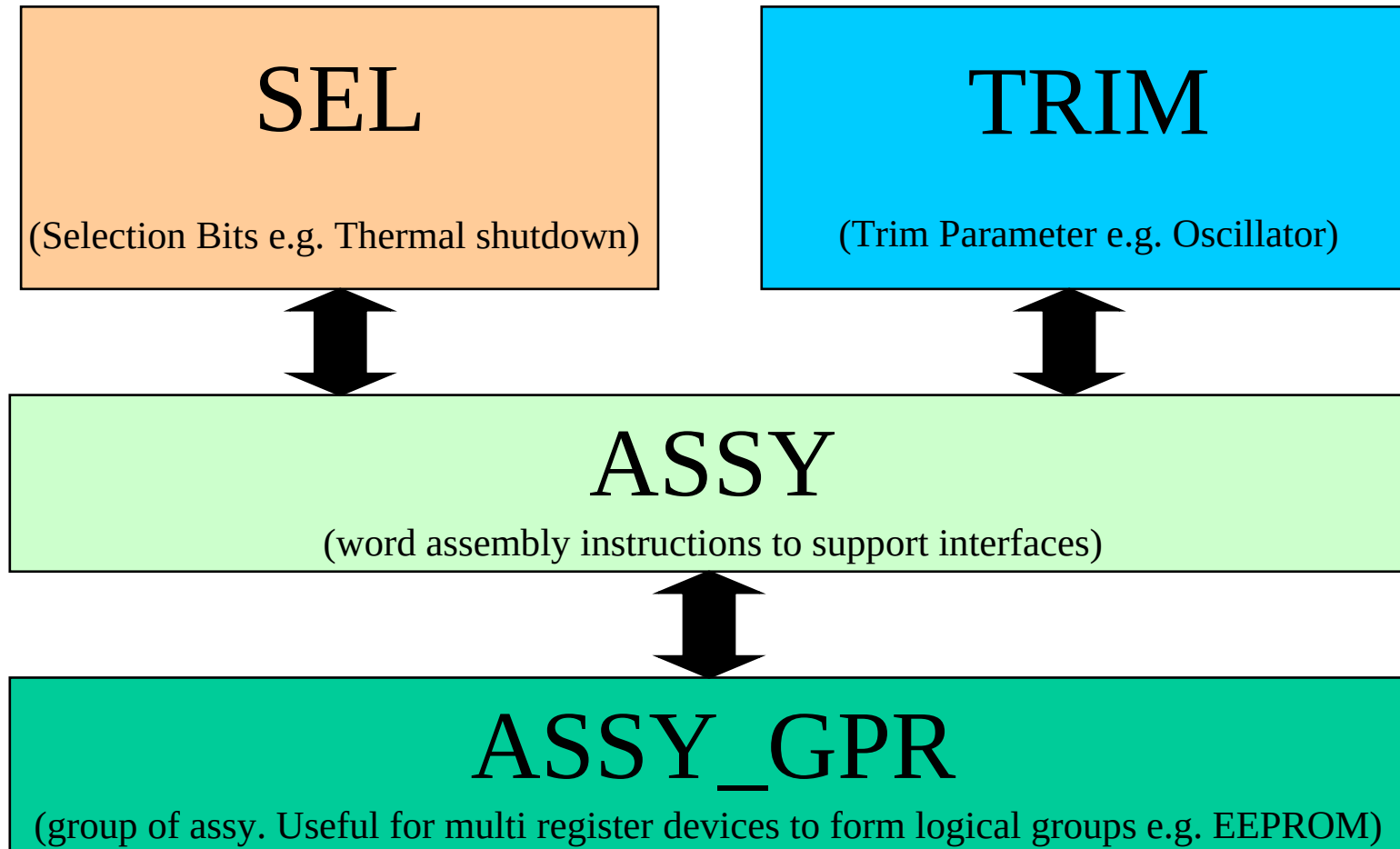
- It is a Trimming class
- It has Word assembly capability
- It has Word disassembly capability
- It's easy to setup (through a TXT-file)
- It's easy to use
- It has build in debug capabilities
- It can adjust the default (trim start) bits
- It can adjust the trim table on the fly
- It supports trim table char of the first x units
- It is **NOT** a interface (like I²C) but it simplifies the coding of interfaces



Basic Concept



Basic Concept

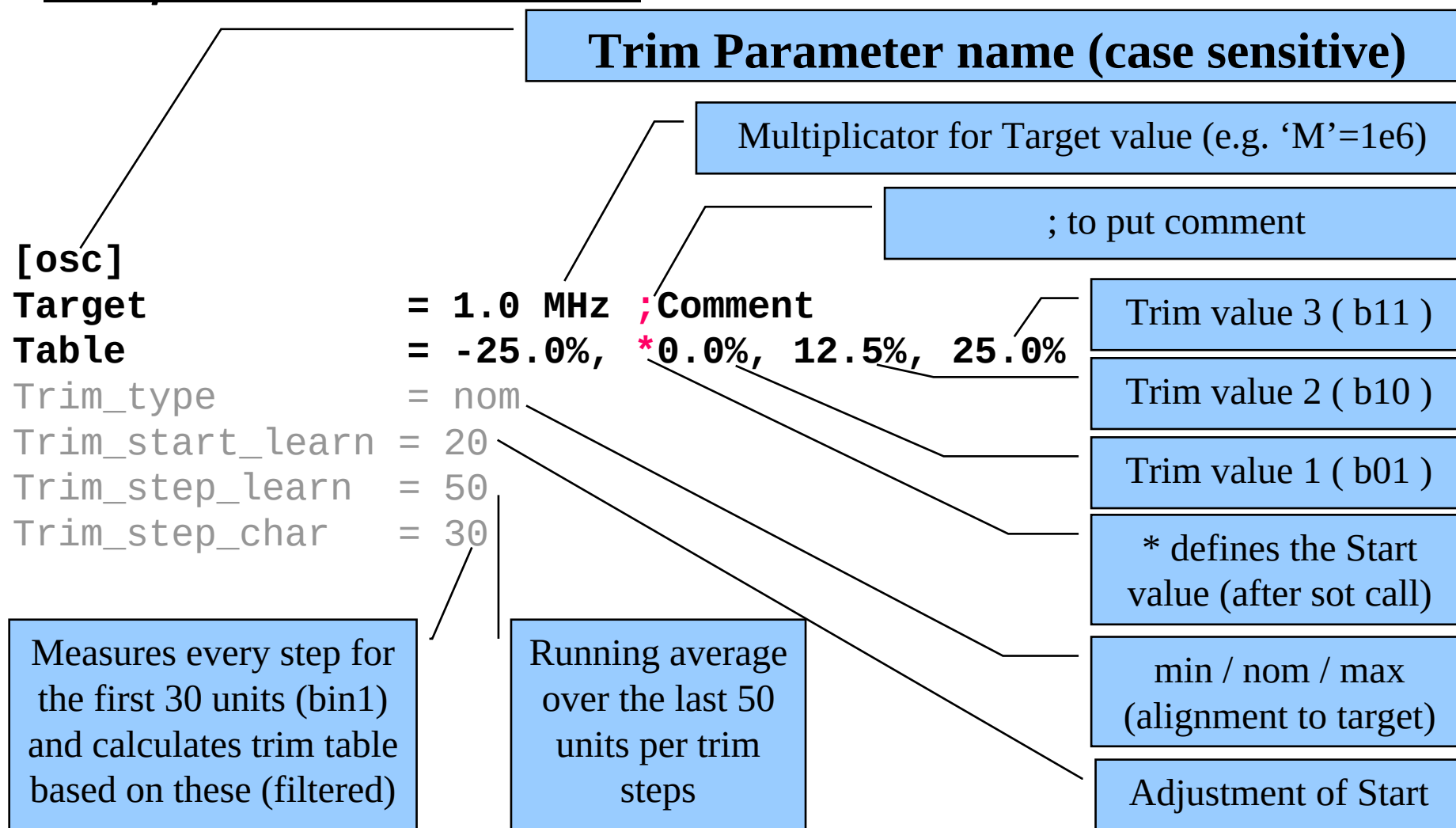




TRIM

Basic Concept

Setup of TRIM in INI-File:

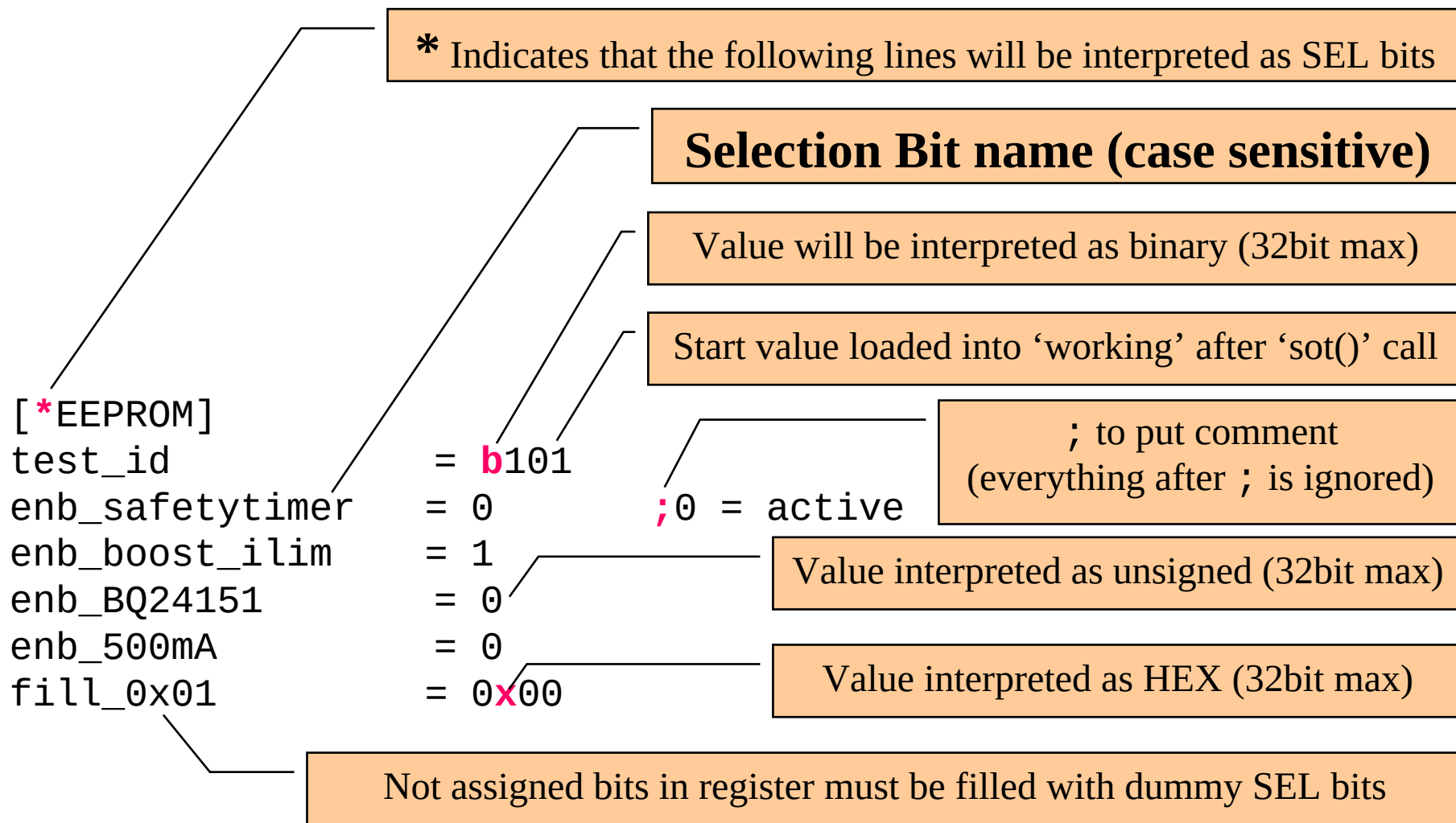




SEL

Basic Concept

Setup of SEL in INI-File:





ASSY

Basic Concept

Setup of ASSY in INI-File:

_ Indicates Assy definition (it will be ignored in Register name)

Assy name (case sensitive)

Name from TRIM or SEL BITS

! Indicates that the value of the bit will be inverted

Bit0 of SEL BITS “test_id” will be linked to REGISTER “reg_0x01” Bit0

“bandgap” BIT0 is linked to “reg_0x01” BIT4

“bandgap” BIT2 is linked to “reg_0x01” BIT5

“bandgap” BIT1 is linked to “reg_0x01” BIT6

Line number = Assy bit pos

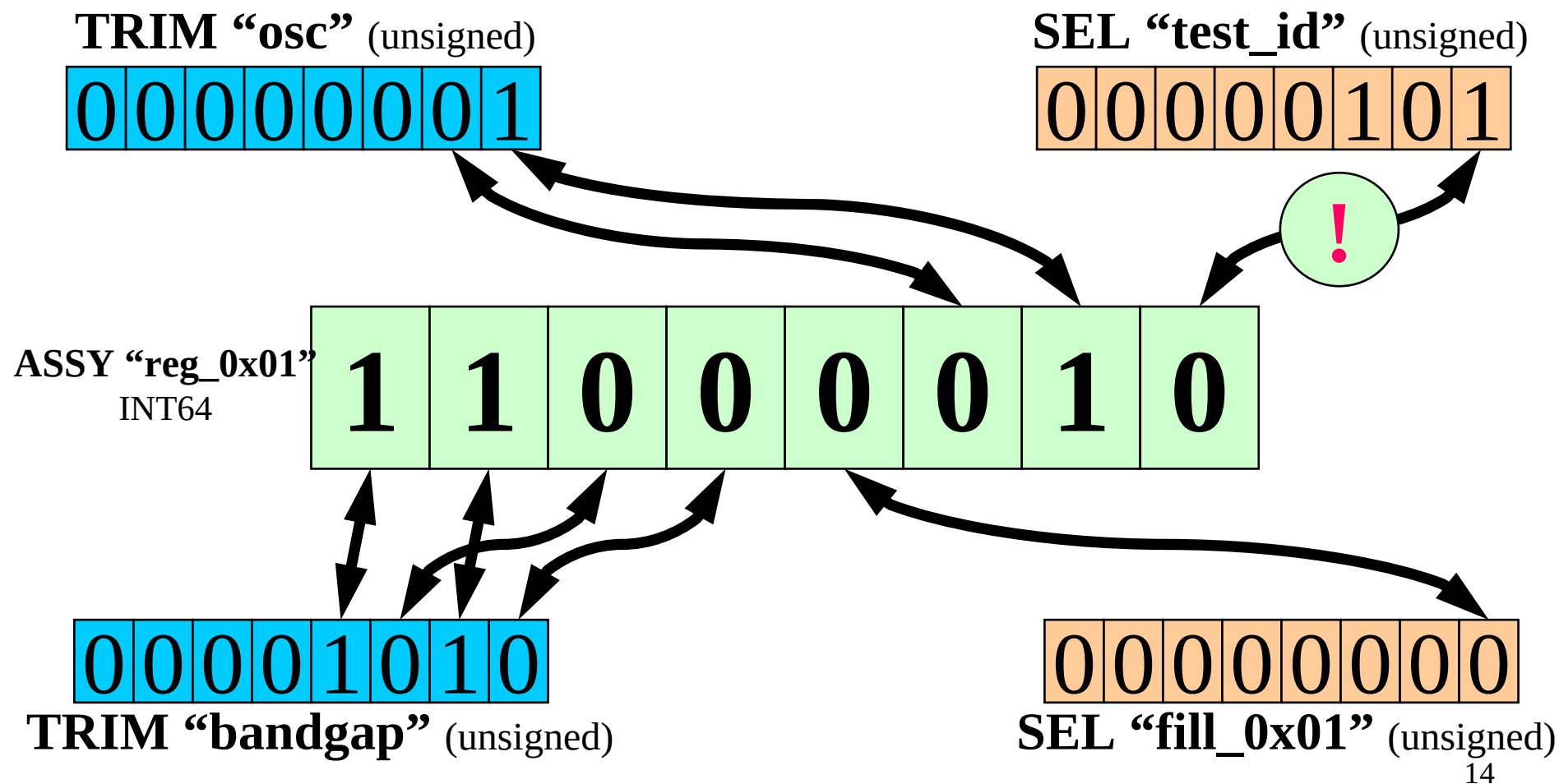
	[_reg_0x01]	
0:	test_id	= !0
1:	osc	= 0
2:	osc	= 1
3:	fill_0x01	= 0
4:	bandgap	= 0
5:	bandgap	= 2
6:	bandgap	= 1
7:	bandgap	= 3



ASSY

Basic Concept

Example of Word assembly



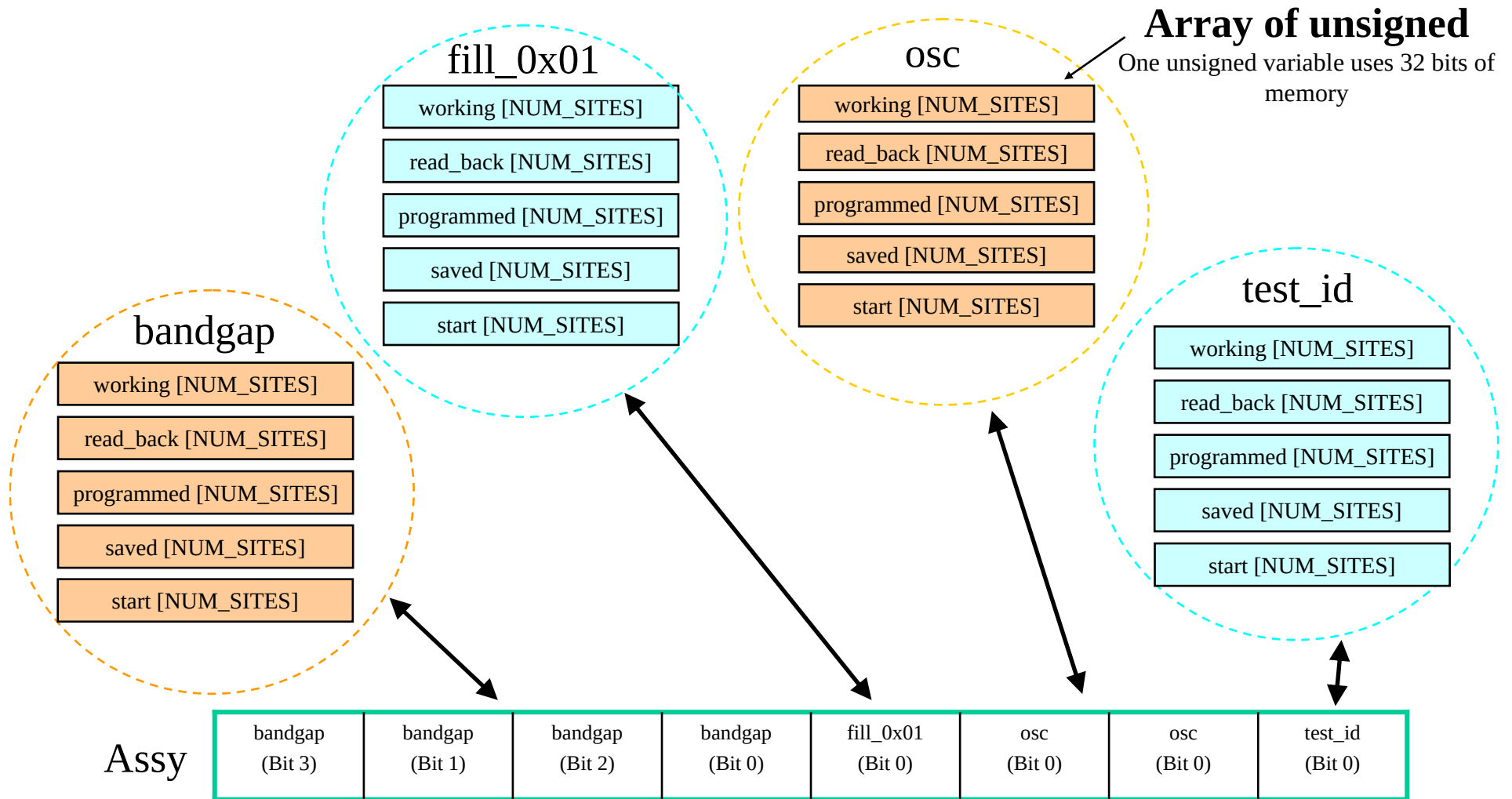


Data Storage

- Information is only stored in TRIM and SEL
- ASSY and ASSY_GRP only have links to TRIM and SEL
- Data requested from ASSY or ASSY_GRP will be assembled just in time
- Data passed into ASSY or ASSY_GRP will be disassembled and stored in TRIM or SEL
- Every SEL or TRIM parameter has multiple storage places
 - **Working** [NUM_SITES] // contains data that should be send to device
 - **Programmed** [NUM_SITES] // used to store data which was burned into EEPROM
 - **Read_back** [NUM_SITES] // used to store data read from device
 - **Saved** [NUM_SITES] // temp storage location
 - **Start** [NUM_SITES] // stores default values



Data storage locations



16



ASSY_GRP

Basic Concept

Setup of ASSY_GRP in INI-File:

Indicates Assy group definition (it will be ignored in Assy Group name)

Assy group name (case sensitive)

Name of ASSY

Register address (integer value for interface)

[#soft_trim]

0: reg_0x01

1: reg_0x02

2: reg_0x03

3: reg_0x04

4: reg_0x05

= 0x01

= 0x02

= 0x03

= 0x04

= 0x05

"write_normal_0x01"

"write_normal_0x02"

"write_normal_0x03"

"write_normal_0x04"

"write_normal_0x05"

String that can be used as Vector Label
(optional)

Line number = position in grp



Simple *.ini file example

```

; ===== TRIM =====
[bandgap]
Target          = 1.233 V
Table           = *0%,-0.329%,-0.687%,-1.015%,-1.405%,-1.732%,-2.091%,-2.418%, 2.806%, 2.477%, 2.120%, 1.790%, 1.402%, 1.072%, 0.714%, 0.386%
Trim_start_learn = 20 ; changes Default after one trim step was used more then x times
Trim_step_learn  = 50 ; runnig average of the step width of the last x units
Trim_step_char   = 30 ; enables trim table char for x GOOD units

[osc]
Target          = 1.0 MHz
Table           = -25.0%, *0.0%, 12.5%, 25.0%
Trim_type       = nom
Trim_start_learn = 20 ; changes Default after one trim step was used more then x times
Trim_step_learn  = 50 ; runnig average of the step width of the last x units
Trim_step_char   = 30 ; enables trim table char for x GOOD units

; ===== SEL =====
[*EEPROM]
test_id          = b101 ;later on used for check-sum
enb_safetytimer  = 0 ;0 = active
enb_boost_ilim   = 0
enb_BQ24151      = 0
enb_500mA        = 0
fill_0x01        = 0x00

; ===== Assy =====
[_reg 0x01]
0: test_id       = 0
1: osc           = 1
2: osc           = 2
3: fill_0x01     = 0
4: bandgap       = 0
5: bandgap       = 1
6: bandgap       = 2
7: bandgap       = 3

; ===== Assy_grp =====
[#soft_trim]
0: reg_0x01 = 0x01 "write_normal_0x01"
1: reg_0x02 = 0x02 "write_normal_0x02"
2: reg_0x03 = 0x03 "write_normal_0x03"
3: reg_0x04 = 0x04 "write_normal_0x04"
4: reg_0x05 = 0x05 "write_normal_0x05"

```

18



Class Setup



ef1553.h

```
#include "treg/treg.hxx"  
extern      TREG  dut;
```

ef1553.cpp

```
TREG  dut;           // create global instance of TREG  
  
BOOL Cef1553::OnTestInit( void ) {  
    dut.init("file_name.ini", NUM_SITES); // initializes TREG  
}  
  
BOOL Cef1553::OnTestStartup( void ) {  
    dut.sot();        // initializes parameters for new test run  
}  
  
BOOL Cef1553::OnTestCompletion( void ) {  
    dut.eot();        // required for adaptive trim learning  
}
```

20



How to use



How to use

`dut.`

- assy
- assy_grp
- eot
- file
- find
- init
- reader
- sel
- sot
- trim

`dut.trim("osc").|`

- get_read_back
- get_saved
- get_start
- get_steps
- get_table_value
- get_target
- get_trim_allowed
- get_working**
- get_written
- get_written_valid

receive working values from class

`[osc]`

Target	= 1.0 MHz
Table	= -25.0%, *0.0%, 12.5%, 25.0%

`[reg_0x01]`

0: test_id	= 0
1: osc	= 1
2: osc	= 2
3: fill_0x01	= 0
4: bandgap	= 0
5: bandgap	= 1
6: bandgap	= 2
7: bandgap	= 3

`dut.assy("reg_0x01").|`

- add_new_bit
- assembly_name
- assy_bit_root
- comp_prog
- comp_prog_to_read
- comp_read
- comp_read_to_start
- comp_written_to_working
- copy_read_to_prog
- count

`dut.assy_grp("soft_trim")[i].|`

access every ASSY like in an array (based on line number)

- add_new_bit
- assembly_name
- assy_bit_root
- comp_prog
- comp_prog_to_read
- comp_read
- comp_read_to_start
- comp_written_to_working
- copy_read_to_prog
- count

`[#soft_trim]`

0: reg_0x01	= 0x01
1: reg_0x02	= 0x02



Use of “Short Cuts”

How to use

```
TRIM_NODE      &bg      = dut.trim("bandgap");      //Reference to bandgap
SEL_NODE       &t_id     = dut.sel("test_id");      //Reference to test_id
ASSY_NODE      &reg_0x01 = dut.assy("reg_0x01");    //Reference to reg_0x01
ASSY_GRP_NODE  &soft_trim = dut.assy_grp("soft_trim"); //Reference to soft_trim
```

bg.

- calc_abs_step_estimate
- calc_rel_step_estimate
- eot
- find_best_trim_step
- get_guessed_final
- get_learn_table_value
- get_node_name
- get_post_reading
- get_pre_reading
- get_programmed

t_id.

- get_node_name
- get_programmed
- get_read_back
- get_saved
- get_start
- get_working
- get_written
- get_written_valid
- next_sel_node
- pre_sel_node

reg_0x01.

- add_new_bit
- assembly_name
- assy_bit_root
- comp_prog
- comp_prog_to_read
- comp_read
- comp_read_to_start
- comp_written_to_working
- copy_read_to_prog
- count

reg_0x01[bit_pos].

- assy_mask
- get_param_name
- get_programmed
- get_read_back
- get_saved
- get_start
- get_working
- invert_bits
- link
- next_bit

soft_trim[i].

- add_new_bit
- assembly_name
- assy_bit_root
- comp_prog
- comp_prog_to_read
- comp_read
- comp_read_to_start
- comp_written_to_working
- copy_read_to_prog
- count

soft_trim[i][bit_pos].

- assy_mask
- get_param_name
- get_programmed
- get_read_back
- get_saved
- get_start
- get_working
- invert_bits
- link
- next_bit



That's how we trim a parameter...

```
// Test Function: oscillator_trim
ETS_PRGFLOW_FUNC oscillator_trim( int DSIndex, LPCTSTR TestLabel )
{
    int          tnum = 2030;
    RESULTS_STR  freq_value[NUM_SITES];
    TRIM_NODE    &osc = dut.trim("osc");           // "Short cut" to "osc" in TREG
    unsigned     reg_value[NUM_SITES];

    // setup measurement here ----

    SERIAL {
        reg_value[site] = dut.assy("reg_0x01").get_working(site); // get content of 'working' from ASSY 'reg_0x01'
        i2c.write(0x01, reg_value[site]);                       // write reg_value into device
    }
    meas_frequency(0.5, 10, 300, freq_value);                  // measure frequency

    osc.pre(freq_value);                                       // pass frequency values into TREG to trim devices
                                                                // -> this changes the content of dut.trim("osc") -> working[NUM_SITES]

    TEST(tnum, freq_value);                                    // test default frequency values against wide limits

    SERIAL {
        reg_value[site] = dut.assy("reg_0x01").get_working(site); // get updated content of 'working' from ASSY 'reg_0x01'
        i2c.write(0x01, reg_value[site]);                       // write reg_value into device (contains new trim settings)
    }
    meas_frequency(0.5, 10, 300, freq_value);                  // measure frequency with updated trim settings

    osc.post(freq_value);                                       // pass trimmed frequency values into TREG -> for trim learning

    TEST(tnum, freq_value);                                    // test trimmed frequency values against tight limits
}
}
```

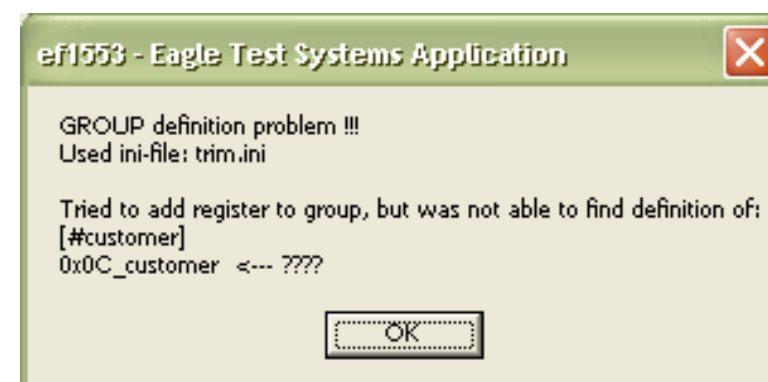
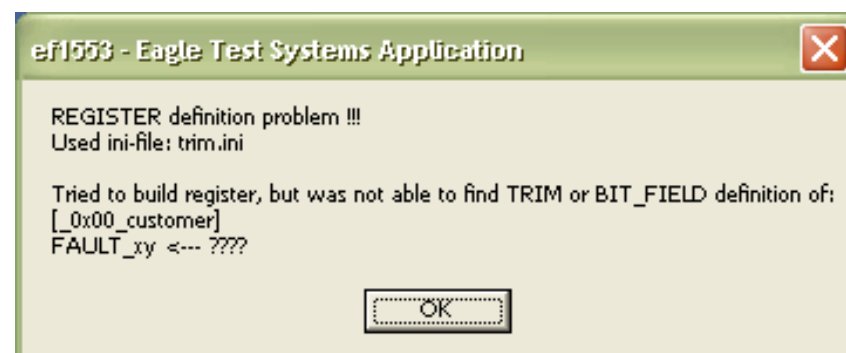
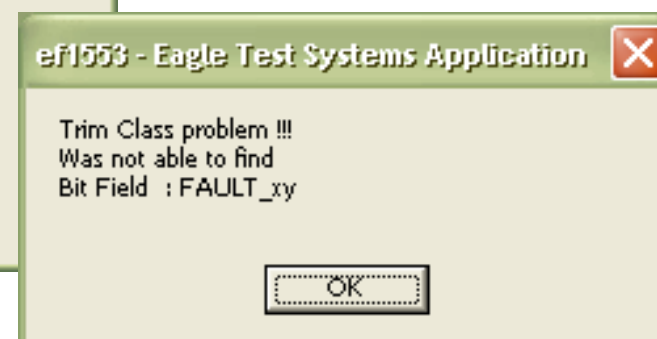
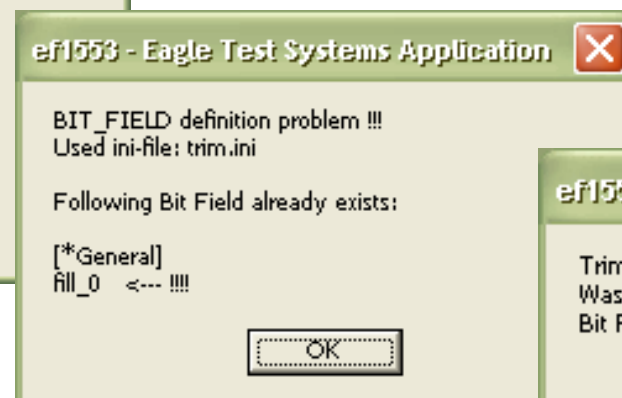
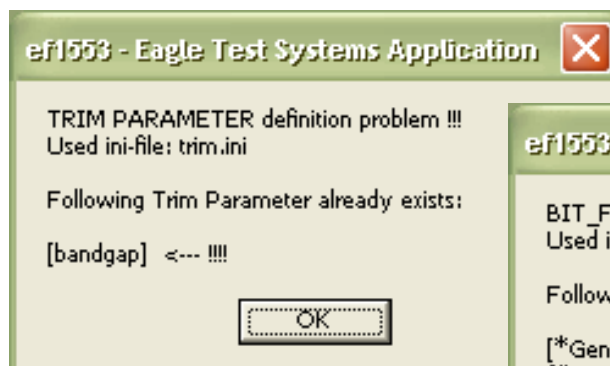
Trim here

Learn here

Important: The execution of osc.pre(freq_value) changes the content of 'working' !!!!

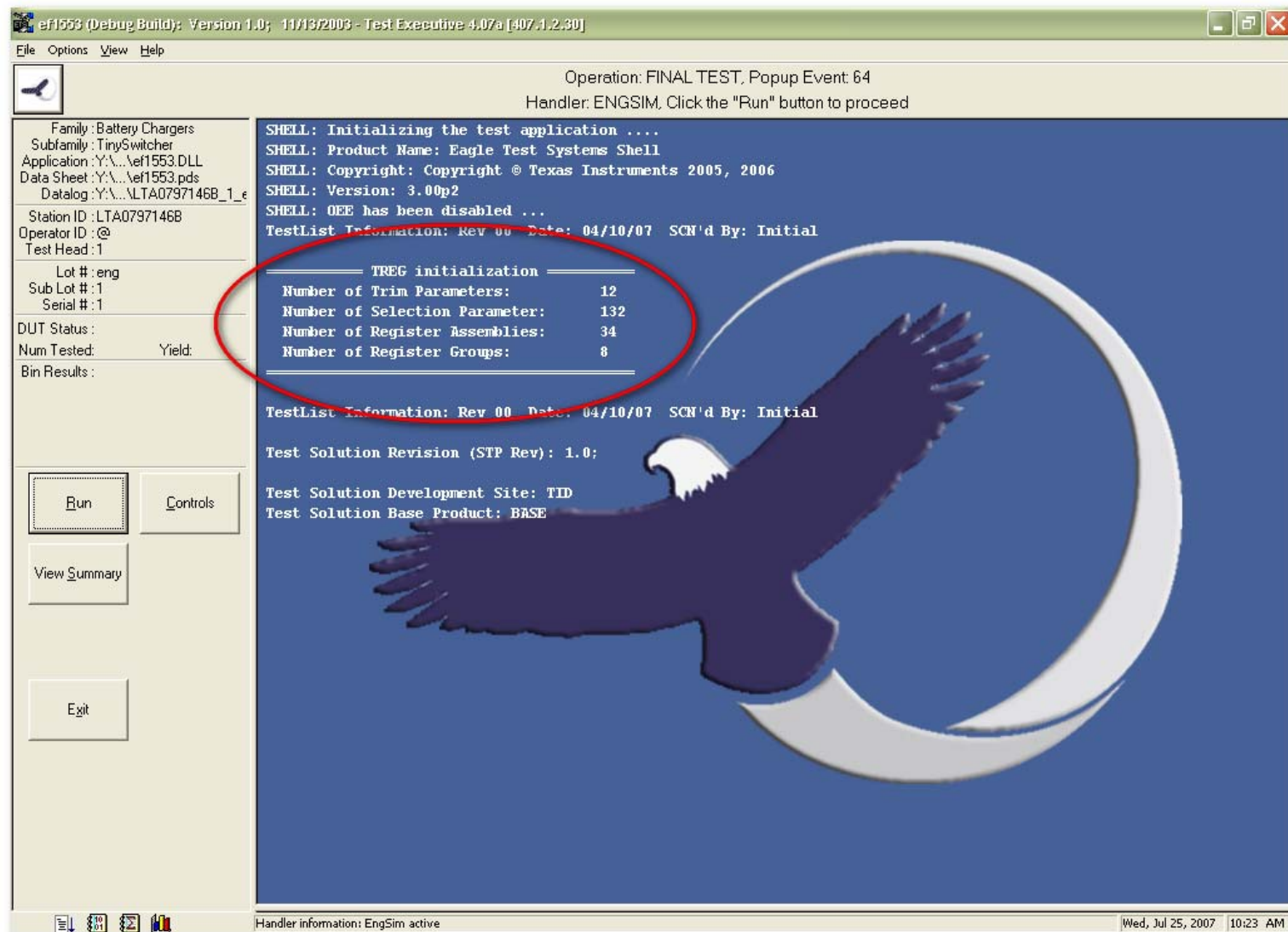


Debug tools





Debug Tools



27



Check of current status of all assy

`dut.assy.print(site);` // goes into "register.txt"

Register Name	Reg_size	Pos	in Reg	Parameter Name	Param Bit	Site	Work	Prog	Read	Start	Saved
reg_0x01	8	0		test_id	!0	0	1	1	0	1	1
reg_0x01	8	1		osc	1	0	0	0	0	0	0
reg_0x01	8	2		osc	2	0	1	1	0	1	1
reg_0x01	8	3		fill_0x01	0	0	0	0	0	0	0
reg_0x01	8	4		bandgap	0	0	0	0	0	0	0
reg_0x01	8	5		bandgap	2	0	0	0	0	0	0
reg_0x01	8	6		bandgap	1	0	0	0	0	0	0
reg_0x01	8	7		bandgap	3	0	0	0	0	0	0
						0	0x0005	0x0005	0x0000	0x0005	0x0005

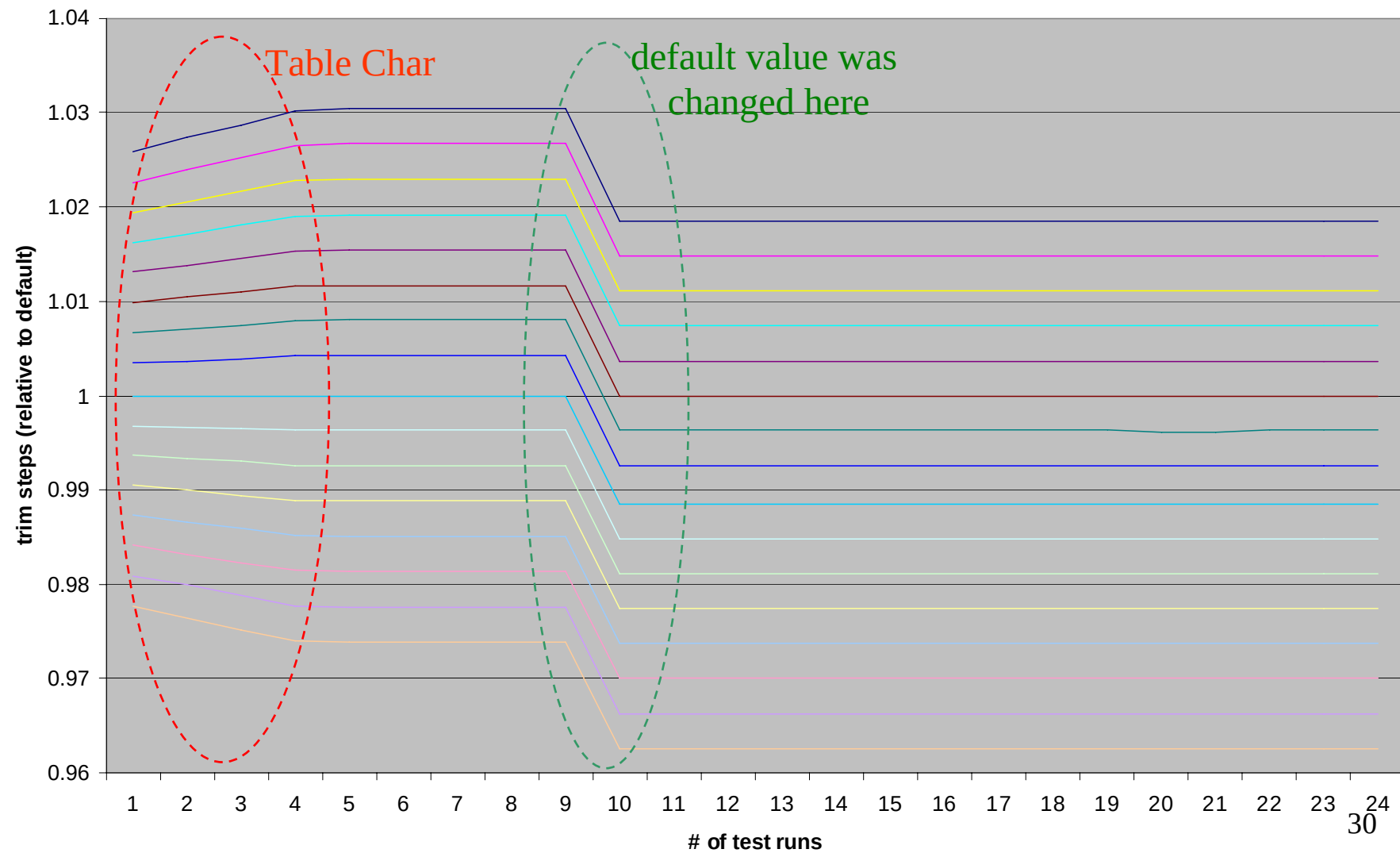
Register Name	Reg_size	Pos	in Reg	Parameter Name	Param Bit	Site	Work	Prog	Read	Start	Saved
0x0D_soft	8	0		test_id	3	0	0	0	0	0	0
0x0D_soft	8	1		test_id	4	0	0	0	0	0	0
0x0D_soft	8	2		enb_boost_ilm	0	0	0	0	0	0	0
0x0D_soft	8	3		enb_buck_hs_ilm	0	0	1	1	0	1	1
0x0D_soft	8	4		enb_buck_ls_ilm	0	0	0	0	0	0	0
0x0D_soft	8	5		enb_3MHz	0	0	0	0	0	0	0
0x0D_soft	8	6		comp_select	0	0	0	0	0	0	0
0x0D_soft	8	7		enb_temp_reg	0	0	0	0	0	0	0
						0	0x0008	0x0008	0x0000	0x0008	0x0008



Adaptive Trim Learning

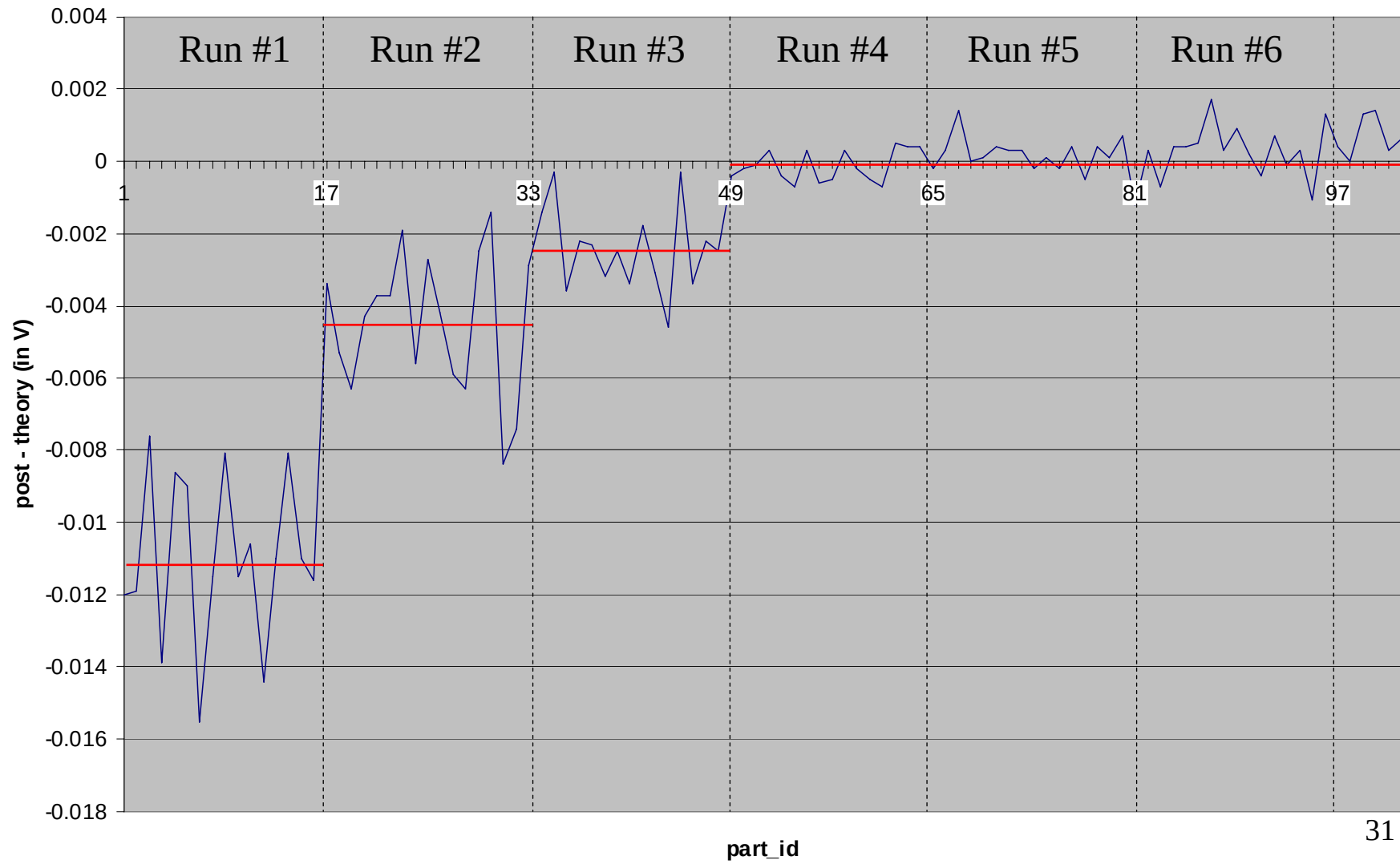


Vref trim table



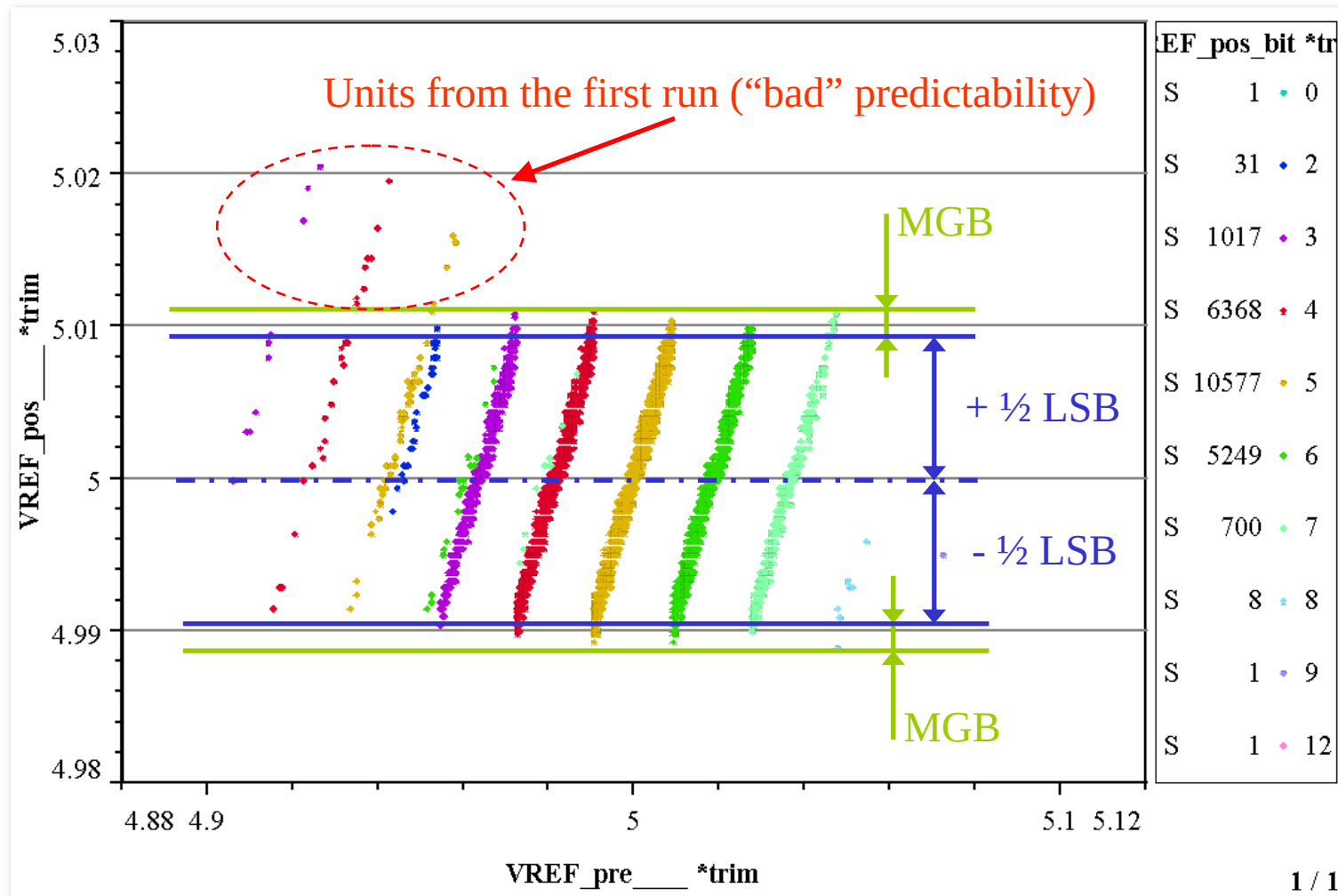


Trim predictability





Trim parameter folding



Note: The bad predictability can be avoided by doing the trim table char before the device trimming (execution of 'pre()').

32

As all the trim steps are measured before the trimming the class can use this values to trim accurately from the first unit on



Advantages of adaptive trim learning

- Adjusts to process shifts
 - More accurate trimming
 - Tighter specs
 - accuracy advantage over competition → higher GPM
- Time savings
 - If the post trim bit setting is the same as the pre trim bit setting there is no need to measure the post trim parameter value as this is the same as the pre trim parameter value
 - Skipping the second measurement can save a lot of time → lower OTC

Allow skip of second measurement	TPS61240 Test time per Wafer
yes	1:10:58
no	1:19:48
0:08:50 = 11%	



How do I set it up?

- add following things in your *.ini file

```
[osc]
Target          = 1.0 MHz
Table           = -25.0%, *0.0%, 12.5%, 25.0%
```

Trim_start_learn = 20 ← delay for default adjust

Trim_step_learn = 50 ← running average over the last 50 units (per step)

Trim_step_char = 30 ← sets flag to enable the measurement of the whole trim table (30 units)

- add table char to your code (before trim)

```
if(CHAR_MODE || osc.table_char_active()){           // check if table char is still active
    osc.save_working();                               // save working values
    for(unsigned step=0; step<osc.get_steps(); step++){ // loop through trim steps
        osc.set_working(step);                       // set working value to loop count
        es.write_grp("preview_tm");                   // send working value to device
        meas_frequency(0.5, 10, 300, osc_res, &SS);   // measure frequency
        TEST(tnum_char+step, osc_res, osc.get_target); // Datalog table (optional)
        osc.table_char(osc_res, step);               // hand over step values to TREG
    }
    osc.restore_working();                             // restore working values
    es.write_grp("preview_tm");                         // send working value to device
}
```



Skip Post trim measurement

```
// Test Function: oscillator_trim
ETS_PRGFLOW_FUNC oscillator_trim( int DSIndex, LPCTSTR TestLabel )
{
    int          tnum = 2030;
    RESULTS_STR  freq_value[NUM_SITES];
    TRIM_NODE    &osc = dut.trim("osc");           // "Short cut" to "osc" in TREG
    unsigned     reg_value[NUM_SITES];

    // setup measurement here ----

    SERIAL {
        reg_value[site] = dut.assy("reg_0x01").get_working(site); // get content of 'working' from ASSY 'reg_0x01'
        i2c.write(0x01, reg_value[site]);                        // write reg_value into device
    }
    meas_frequency(0.5, 10, 300, freq_value);                  // measure frequency

    osc.pre(freq_value);                                       // pass frequency values into TREG to trim devices
                                                                // -> this changes the content of dut.trim("osc") -> working[NUM_SITES]

    TEST(tnum, freq_value);                                    // test default frequency values against wide limits

    if (osc.updated_by_trim()){                                // if osc trim bits of one site are different then the default measure again
        SERIAL {
            reg_value[site] = dut.assy("reg_0x01").get_working(site); // get updated content of 'working' from ASSY 'reg_0x01'
            i2c.write(0x01, reg_value[site]);                        // write reg_value into device (contains new trim settings)
        }
        meas_frequency(0.5, 10, 300, freq_value);                // measure frequency with updated trim settings

        osc.post(freq_value);                                     // pass trimmed frequency values into TREG -> for trim learning

        TEST(tnum, freq_value);                                   // test trimmed frequency values against tight limits
    }
}
```

Important: Make sure the trim parameter will get tested against spec limits !!!
Don't skip 2nd test statement !!!!!!!



Questions ??